

SDEV 2351

DAY 3: STATE CHALLENGES AND FLUX/REDUX PATTERN INTRO

MODULE 1 — ROUTING & STATE

Session Objectives

1. Identify three state management challenges in single-page applications (loss on navigation, prop drilling, synchronization)
2. Explain the Flux/Redux pattern and unidirectional data flow
3. Describe how actions, reducers, and a central store work together to manage application state

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	Day 2 bridge + state loss scenario
Exploring	25 min	Three state challenges, Flux/Redux pattern, demo
Practicing	60 min	Analysis, Flux mapping on paper, group discussion
Assessing	15 min	Knowledge check, exit ticket, Q&A

PHASE 1 · 10 MINUTES

CONNECTING

From navigation to the data problem it creates

The Data Problem

Yesterday you built a checkout wizard -- code that navigates users between pages.

But what happens to the data on those pages when you navigate away?

- Shopping cart: items disappear when you click to another page
- Each page stores data in private fields
- When the component is destroyed, those fields are gone

Today is **pattern-day**, not coding-day. You'll diagram the Flux/Redux cycle on paper. Tomorrow you write real Fluxor code.

PHASE 2 · 25 MINUTES

EXPLORING

Three problems, one pattern

Challenge 1: State Loss on Navigation

Component-level fields reset when the component is destroyed and re-created:

- `private List<CartItem> items` lives inside the Products page
- Navigate to Cart page -- Products component is destroyed
- Navigate back -- a **new** Products component is created with an empty list

Symptom: User adds items, navigates away, comes back -- cart is empty.

Challenge 2: Prop Drilling

Passing data through many component layers via parameters:

- Dashboard → NavBar → CartBadge -- each layer needs a `[Parameter]` just to pass data through
- Adding a new consumer means editing every component in the chain
- Tight coupling between unrelated components

Challenge 3: Cross-Component Synchronization

Multiple components need the same data, but updates in one do not reflect in others:

- `HeaderCartBadge` shows count = 3
- `CartSummary` shows count = 2
- Both read from their own local copy -- no single source of truth

The Flux/Redux Pattern

Unidirectional data flow -- data flows one direction around the cycle:

1. **Component** dispatches an **Action** (what happened)
2. **Reducer** takes current state + action, returns **new state**
3. **Store** holds the single source of truth
4. **Components** subscribe to the store and re-render

Key constraint: Components never modify the store directly -- they dispatch actions. This discipline makes state predictable and debuggable.

Mapping to C# Concepts You Know

FLUX/REDUX TERM	C# EQUIVALENT YOU KNOW
Action	Raising an event
Reducer	Event handler that returns new state (instead of modifying existing state)
Store	Singleton service that holds the data
Subscribe	Event subscription (<code>+=</code>)

Tomorrow (Day 4) these map directly to Fluxor classes: `[FeatureState]`, `[ReducerMethod]`, `IState<T>`.

What Belongs in the Store?

Not every piece of data needs centralization:

STORE STATE

Cart items, user session, search filters -- data that must survive navigation or be shared across components.

LOCAL STATE

Is this dropdown open? Hover state, focus, form validation -- UI state that belongs to one component.

PHASE 3 · 60 MINUTES

PRACTICING

Observe, map, discuss -- no code today

Activity 1: Guided Analysis (15 min)

Observation exercise -- read-only exploration

- Open the analysis project
- Follow the 6 observation steps in the Activity Walkthrough (Day 3 resources on Brightspace)
- Identify: where state is stored, what happens on navigation, why the Cart page shows empty data

You are reading code, not writing it. The goal is to see the state-loss problem in action.

Activity 2: Flux Mapping on Paper (25 min)

Pair work -- draw the pattern

Map the shopping cart to the Flux/Redux cycle on paper or whiteboard:

1. **State shape** -- what data does the store hold?
2. **Action names** -- what events can happen? (AddItem, RemoveItem, ClearCart)
3. **Reducer logic** -- pseudocode for how each action produces new state
4. **Subscribers** -- which components need this data?

The drawing you do today IS your blueprint for tomorrow's code.

Activity 3: Group Discussion (10 min)

Groups of 3-4

Review your Days 1-2 applications and identify 1-2 scenarios where centralized state would help.

Each group shares one scenario in one sentence.

Expected examples: checkout wizard data, product search filters, route history, user preferences.

PHASE 4 · 15 MINUTES

ASSESSING

Summary

Today you learned to:

- 1. Identify three state challenges** -- loss on navigation, prop drilling, and cross-component synchronization
- 2. Explain the Flux/Redux pattern** -- unidirectional data flow where components dispatch actions, reducers produce new state, and the store is the single source of truth
- 3. Map Flux/Redux terms to C#** -- actions are events, reducers are handlers, the store is a singleton service

Exit Ticket

1. Describe the Flux/Redux cycle in your own words -- no diagram, just sentences.
2. Name one piece of data that belongs in the store and one that should stay local in a component. Explain why.

What's Next

Day 4: Hands-on Fluxor Implementation

- Create your first Fluxor store
- Define actions and write reducers in C#
- Wire components to subscribe to state changes

No pre-class reading -- Day 4 builds directly on today's pattern work. Bring your Flux mapping diagram.